

Project Report: AI-Based Resume Skill Gap Analyzer

Report date: 3 October 2026

Project level: Beginner-to-medium college project

Implementation: Python text processing with a Streamlit interface

1. Project objective

The AI-Based Resume Skill Gap Analyzer compares skills found in a text-based resume PDF with skills mentioned in a job description. It presents matching skills, missing skills, a Skill Match Score, and a plain-language recommendation for each missing skill.

Despite the project name, the current implementation does not use machine learning or advanced AI. It uses a predefined skill list and keyword matching. It does not produce a professional ATS score.

2. Project files

File | Purpose | Status

app.py | Main Streamlit application, PDF extraction, skill detection, comparison, and results display | Created/updated

requirements.txt | Required Python packages: Streamlit and PyPDF2 | Created/updated

README.md | Project overview, setup instructions, examples, limitations, and future improvements | Created/updated

sample/sample_job_description.txt | Example job description for manual testing |

Already present; retained

PROJECT_REPORT.md | This report | Created

Other pre-existing workspace items, including Project.py and the New Project folder, were not changed as part of this work.

3. Features implemented

- Upload a resume PDF and paste a job description in the app.
- Extract and combine text from every PDF page using PyPDF2.
- Detect the predefined technical skills without regard to letter case.
- Recognize aliases:
 - ML as Machine Learning
 - AI as Artificial Intelligence
 - JS as JavaScript
 - ReactJS as React
 - NodeJS as Node.js
- Avoid treating the letter C inside ordinary words as the C programming language; distinguish it from C++ and C#.
- Compare skill sets:
 - Matching skills = resume skills intersect required skills.
 - Missing skills = required skills minus resume skills.
- Calculate the Skill Match Score as:

$$\text{Skill Match Score} = (\text{number of matching required skills} / \text{number of required skills}) * 100$$

The result is rounded to one decimal place. No score is shown when no recognizable skills are found in the job description.

- Categorize scores as Strong (80-100%), Good (60-79%), Partial (40-59%), or Low (below 40%) Skill Match.
- Generate one learning recommendation for each missing skill.
- Show friendly messages for missing input, unreadable or invalid PDFs, PDFs

with no extractable text, and descriptions without recognized skills.

The recognized skills are kept in the editable SKILLS list in app.py.

4. Technologies and dependencies

- Python
- Streamlit
- PyPDF2
- Python standard-library modules: io and re

The workspace virtual environment already contained Streamlit 1.65.0 and PyPDF2 3.0.1. No package installation was necessary during this work.

No database, authentication, paid API, web framework other than Streamlit, machine-learning package, or external AI service is used.

5. Validation performed

The following checks were performed:

1. Syntax check: app.py compiled successfully with Python's py_compile.
2. Helper tests: Five tests passed for:
 - Alias and common-skill detection.
 - Correct detection of standalone C and avoidance of false matches inside words or C++/C#.
 - Match-score calculation and empty required-skill handling.
 - Recommendation generation.
 - Invalid PDF handling and empty PDF text extraction.

Streamlit smoke test: The app page and health endpoint both responded with HTTP 200 on port 8502.

4. Browser workflow test: A temporary text-based test PDF was uploaded, a job description was entered, and Analyze Resume displayed results.

The test showed 9 resume skills, 7 required skills, 3 matching skills, 4 missing skills, and a 42.9% Skill Match Score. The temporary test PDF was removed afterward.

5. Port handling: Port 8501 was already occupied, so the test server used port 8502. The existing process on 8501 was left untouched.

The initial invalid PDF used in the browser flow correctly produced the requested unreadable-PDF message; a valid temporary PDF was then used to complete the successful end-to-end browser test.

6. Running the project

Open a terminal in the project folder and install dependencies if needed:

```
powershell
python -m pip install -r requirements.txt
```

Start the Streamlit application:

```
powershell
python -m streamlit run app.py
```

If python is not available on Windows PATH but the workspace virtual environment is present, run:

```
powershell
.\venv\Scripts\python.exe -m streamlit run app.py
```

Streamlit prints the address and port it selected. To stop the server, press Ctrl+C in its terminal.

7. Browser access and deployment status

During the latest check, the app responded at:

- <http://localhost:8502>
- <http://127.0.0.1:8502>

These are local addresses and work on the computer running the app while its Streamlit server is running. They are not permanent hosted links.

The app has not been deployed to a public hosting service. An address such as <http://192.168.1.7:8502> may work for another device on the same local network, subject to firewall and network settings; that address can change and is not a public deployment.

8. Example analysis

Example job description:

text

We are looking for a Python Developer with experience in Python, SQL, Git, Docker and AWS. Knowledge of REST API and Machine Learning is an advantage.

With a sample resume containing Python, SQL, Git, HTML, CSS, JavaScript, Pandas, Django, and MySQL, the application reports:

- Resume skills detected: 9
- Required skills detected: 7
- Matching skills: Python, SQL, Git
- Missing skills: Docker, AWS, Machine Learning, REST API
- Skill Match Score: 42.9% Partial Skill Match

Recommendations are displayed for each of Docker, AWS, Machine Learning, and REST API.

9. Limitations

- Keyword matching does not understand context or related concepts.
- Only skills in the predefined list and supported aliases are recognized.
- Scanned PDFs made of images generally have no extractable text.
- The app does not evaluate skill proficiency, experience quality, or resume sections.
- The app does not save resumes or results to a database.
- The app is a local Streamlit application unless separately deployed.

10. Suggested future improvements

- Extend the editable skills list with carefully chosen aliases.
- Add support for additional document formats.
- Provide visual comparison of resume and job skills.
- Add resume section detection and more detailed, explainable feedback.
- Deploy through a hosting service if a stable public link is needed.